



**TEHNIŠKI ŠOLSKI CENTER MARIBOR
SREDNJA STROJNA ŠOLA
Zolajeva ulica 12
2000 Maribor**

AVTOMATSKI HRANILNIK ZA DOMAČE ŽIVALI

Izobraževalni program:

tehnik mehatronike

Šolsko leto 2023/24

Mentor:

Izdelal:

Tilen Jäger

Letnik in oddelek: 4.b/1

Maj, 2024

KAZALO VSEBINE

KAZALO VSEBINE	III
KAZALO SLIK	IV
KAZALO TABEL	V
KAZALO PRILOG	VI
UPORABLJENE KRATICE	VII
1 UVOD	1
2 IDEJA	2
2.1 DELOVANJE	2
2.2 NAČRTOVANJE	3
2.2.1 IDEJNA SKICA	3
2.2.2 3D MODEL	4
2.2.3 IZDELAVA OHIŠJA	5
2.3 KOMPONENTE	8
2.3.1 NodeMCU ESP8266 (MIKROKRMILNIK)	8
2.3.2 OSTALE KOMPONENTE	10
2.4 ELEKTRIČNO VEZJE	11
2.5 PROGRAMSKA KODA	12
2.6 PROGRAMIRANJE MIKROKRMILNIKA	12
2.6.1 PROGRAMSKA OPREMA ESP 8266 V ARDUINU	15
2.7 PROGRAMIRANJE SPLETNE APLIKACIJE	18
3 ZAKLUČEK.....	19
4 VIRI.....	20
5 PRILOGE.....	21

KAZALO SLIK

Slika 1: Idejna skica	3
Slika 2: 3D model	4
Slika 3: Krmilni del avtomatskega hranilnika.....	5
Slika 4: Propeler avtomatskega hranilnika.....	6
Slika 5: Zbiralna posoda avtomatskega hranilnika.....	6
Slika 6: Pokrov za zakrivanje električnih krmilnih delov	7
Slika 7: Pokrov za zbiralno posodo.....	7
Slika 8: Prikaz ESP8266 pinov	8
Slika 9: Vezje komponent	10
Slika 10: Električni načrt	11
Slika 11: Vključene knjižnice	12
Slika 12: Prikaz SSID in gesla	12
Slika 13: Prikaz dela kode "Start"	13
Slika 14: Prikaz dela kode "Stop".....	13
Slika 15 Prikaz dela kode "Nastavitev ure"	13
Slika 16: Prikaz izgleda aplikacije S Arduino kode.....	14
Slika 17: Prikaz ročnega nalaganja knjižnice.....	15
Slika 18: Prikaz vpisa spletne strani	16
Slika 19: Prikaz izbire "boards manager"	16
Slika 20: Prikaz instaliranja ESP8266.....	17
Slika 21: Prikaz izbire plošče	17
Slika 22: Prikaz izgleda aplikacije	18

KAZALO TABEL

Table 1: NodeMCU ESP8266, Pini in opis pinov	9
--	---

KAZALO PRILOG

PRILOGA A: Arduino koda

PRILOGA B: Flutter koda

UPORABLJENE KRATICE

IDE – Integrated Development Environment, integrirano razvojno okolje

Wi-Fi – Wireless Fidelity, brezžično omrežje.

Arduino IDE – Programski jezik za programiranje mikrokontrolerov.

Flutter – Programski jezik za programiranje mobilne aplikacije.

Tinkercad – Spletna aplikacija za 3D modeliranje

URL – enolični krajevnik vira (Uniform Resource Locator)

AVR C – Programski jezik

EasyEDA – Spletna stran za izdelavo električnih načrtov

C++ – Programski jezik za Arduino IDE

Flutter – programski jezik za programiranje mobilne aplikacije

Dart – programski jezik za programiranje Flutter – ja

1 UVOD

V sodobnem času, kjer se tehnologija nenehno razvija, se pojavlja vedno več potreb po inovativnih rešitvah za olajšanje vsakdanjega življenja. Ena izmed ključnih področij, ki zahteva napredne in avtomatizirane pristope, je skrb za domače živali. V tem kontekstu se predstavlja projekt avtomatiziranega hranilnika za domače živali, ki združuje tehnologijo 3D tiskanja, mikrokrmilnik Arduino (z uporabo NodeMCU ESP8266), DC motor za doziranje hrane ter spletno stran s priročnim vmesnikom za oddaljeno upravljanje.

Namen tega projekta je ustvariti napravo, ki lastniku omogoča nadzor nad prehranjevanjem njihovih ljubljencev, ne glede na to, kje se nahajajo. Z uporabo mikrokrmilnika NodeMCU ESP8266, in 3D tiskanje, se bomo lotili oblikovanja sistema, ki bo omogočala kontroliranje hrane z mobilno aplikacijo.

S tem projektom želimo ponuditi celovito rešitev za lastnike domačih živali, ki si želijo poenostaviti in izboljšati proces hranjenja svojih ljubljencev.

2 IDEJA

Idejo za avtomatizirani hranilnik za domače živali sem dobil, kadar sem opazoval, kako tehnologija spreminja naš vsakdan. Idejo se mi je porodila, ko sem razmišljal o svojem ljubljencu na morju in če ima hrano. Želel sem ustvariti sistem, ki bo omogočal oddaljen nadzor prehranjevanja hišnih ljubljencev preko mobilne aplikacije, kar bi bilo priročno za lastnike v vsakdanjem življenju.

2.1 DELOVANJE

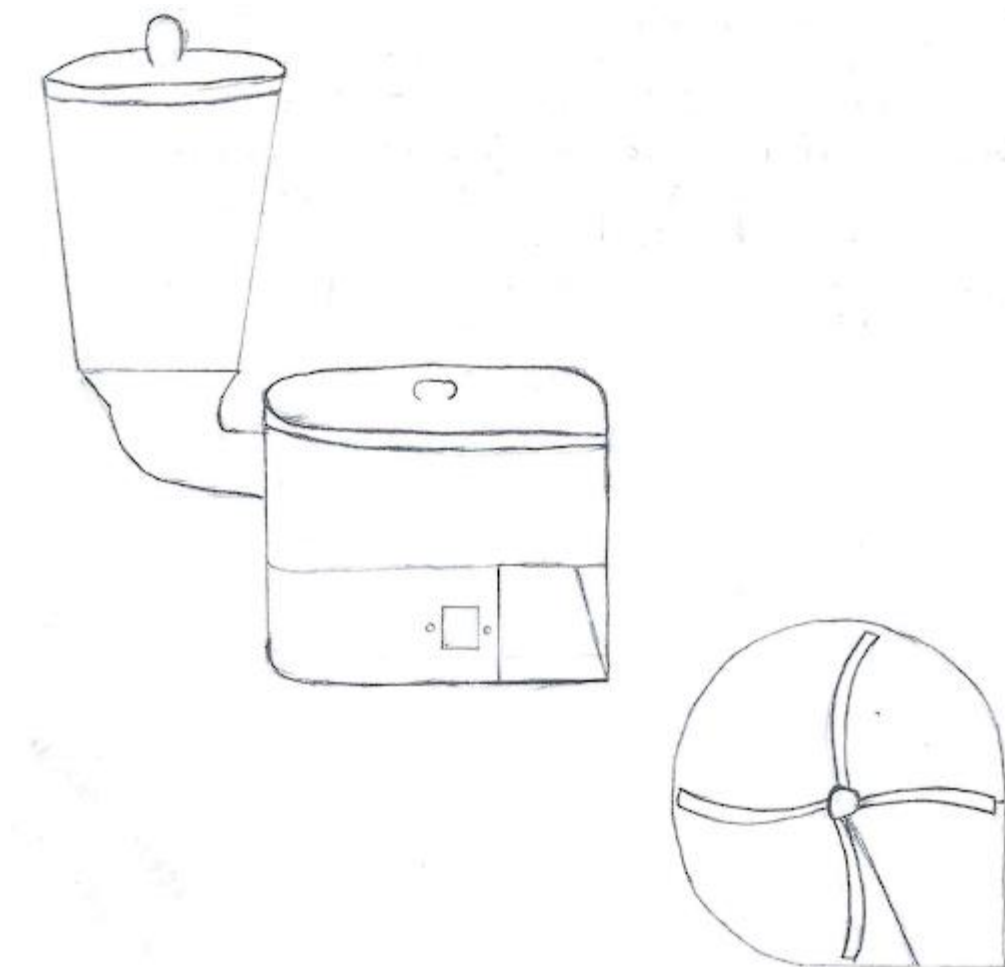
Projekt deluje na principu komunikacije med mikrokrmilnikom z povezavo Wi - Fi-ja in uporabo mobilne aplikacije. Ob zagonu sistema, NodeMCU ESP8266 vzpostavi povezavo z lokalnim omrežjem in pridobi svoj IP naslov, ki ga nato vpišemo v mobilno aplikacijo, s katero nadziramo delovanje hranilnika. Doziranje hrane se lahko aktivira s aktivacijskim gumbom ali s podano uro, ki smo jo nastavili v aplikaciji.

2.2 NAČRTOVANJE

2.2.1 IDEJNA SKICA

Avtomatiziran hranilnik za domače živali je rešitev za učinkovito in enostavno oskrbo s hrano. Glavna posoda, dimenzij 200x200 mm, vsebuje prostor za prenašanje hrane.

Sredi posode se nahaja prostor za prenos hrane, kjer deluje propeler, pritrjen na vrh DC motorja. Ta sistem omogoča natančno doziranje hrane in zagotavlja, da se hrana premika skozi posodo do posode za hrano. (Slika 1)



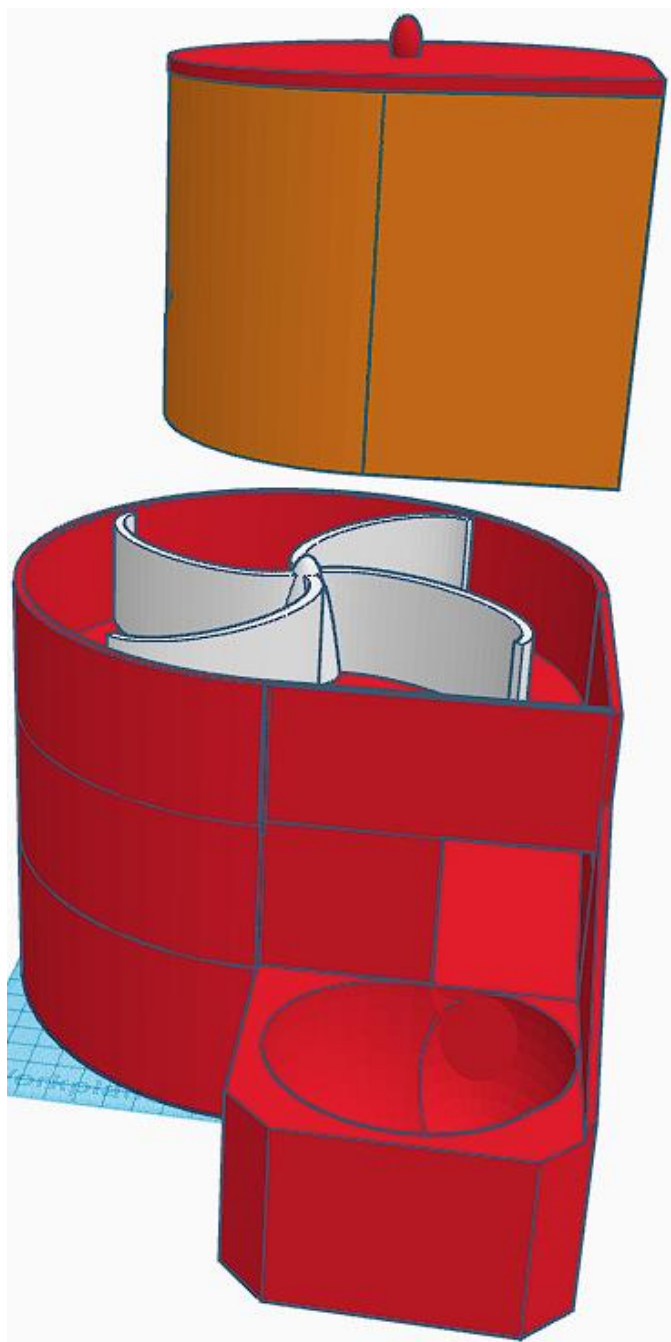
Slika 1: Idejna skica

Vir : Lastni, 2024, 1

2.2.2 3D MODEL

S pomočjo orodja Tinkercad sem uspešno oblikoval 3D model avtomatiziranega hranilnika za domače živali, ki sledi osnovnim načelom iz idejne skice (Slika 1).

Po oblikovanju tridimenzionalnega modela, sem uspešno natisnil končni izdelek s pomočjo PLA filamenta in 3D tiskalnika. Izbral sem filament tipa PLA (polilaktid), zaradi njegove karakteristike in nizke cene. (Slika 2)



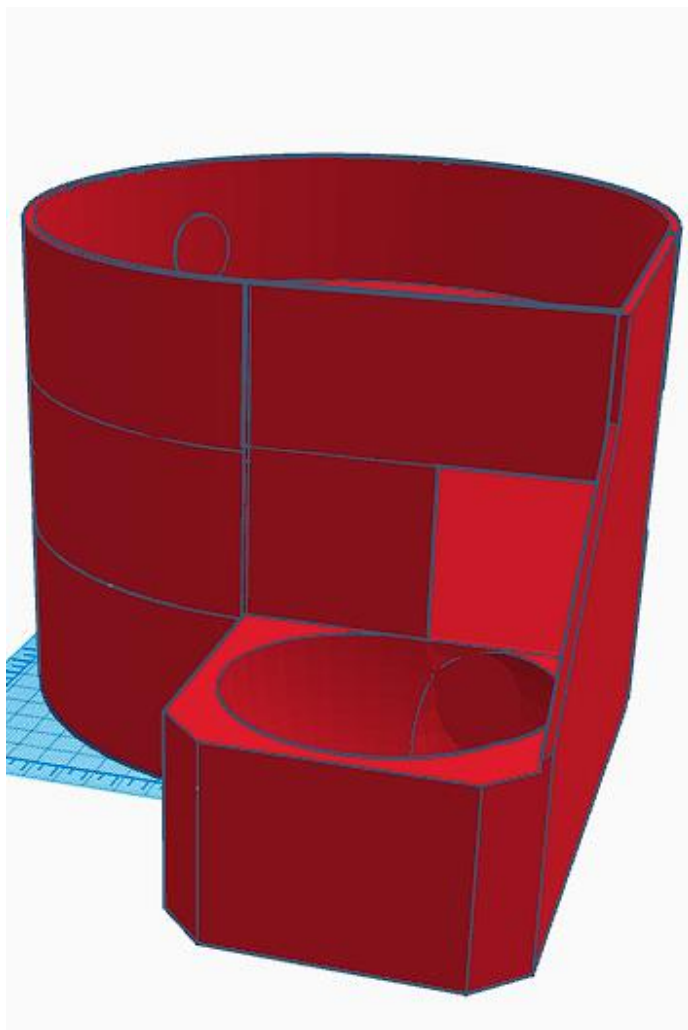
Slika 2: 3D model

Vir : Lastni, 2024, 1

2.2.3 IZDELAVA OHIŠJA

Ohišje je osrednji del avtomatiziranega hranilnika, zgrajeno iz treh glavnih komponent:

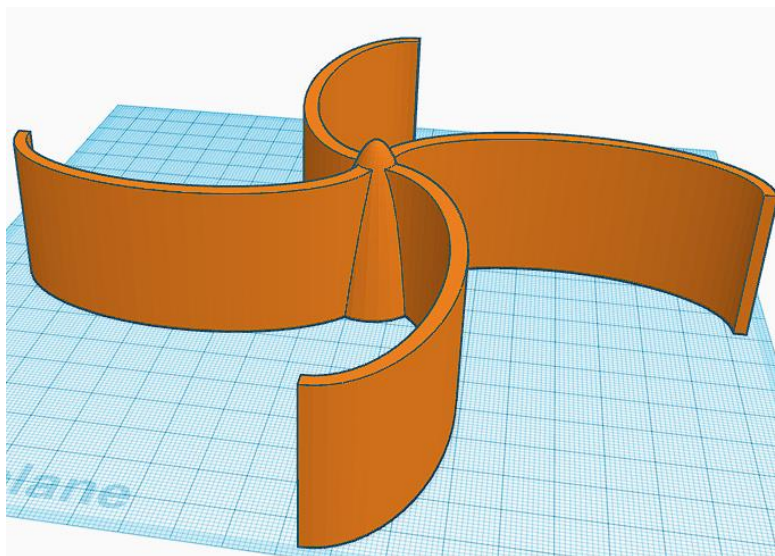
- Prva je krmilni del, ki vsebuje elektronske komponente, potrebne za nadzorovanje sistema, kot je mikrokrmilnik in krmilnik motorja. Je tudi vmesna posoda, skozi katero potuje hrana do hranilne posode. (Slika 3)



Slika 3: Krmilni del avtomatskega hranilnika

Vir: Lasten, 2024, 5

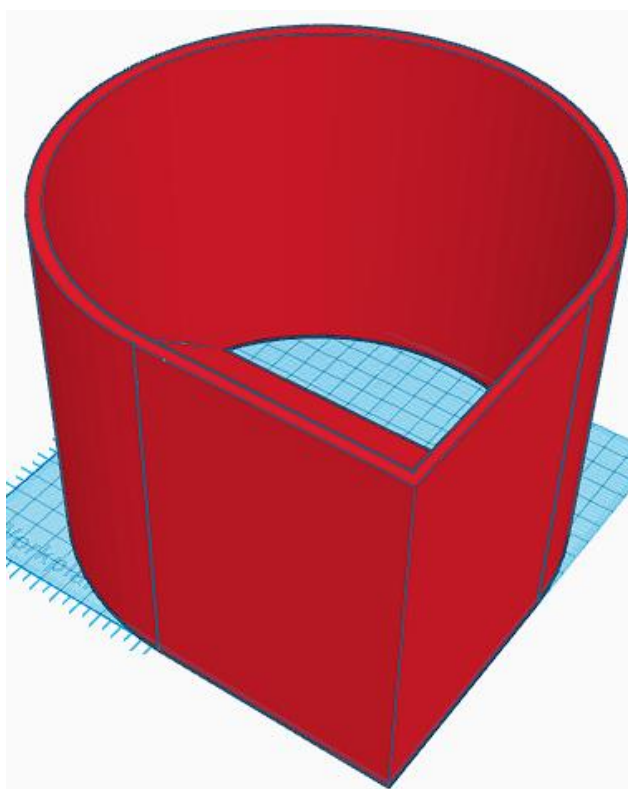
- Druga komponenta je propeler, ki je odgovoren za premikanje hrane iz zbiralne posode skozi vmesne posode do hranilne posode. (Slika 4)



Slika 4: Propeler avtomatskega hranilnika

Vir: Lasten, 2024, 5

- Zadnji del ohišja je zbiralna posoda, katera je zalogovnik za hrano. (Slika 5)



Slika 5: Zbiralna posoda avtomatskega hranilnika

Vir: Lasten, 2024, 5

Poleg teh treh osnovnih delov ima ohišje še dva pokrova:

- Prvi pokrov služi za zakrivanje elektronskih krmilnih delov, kar zagotavlja varno in zaščiteno okolje za občutljive komponente. (Slika 6)

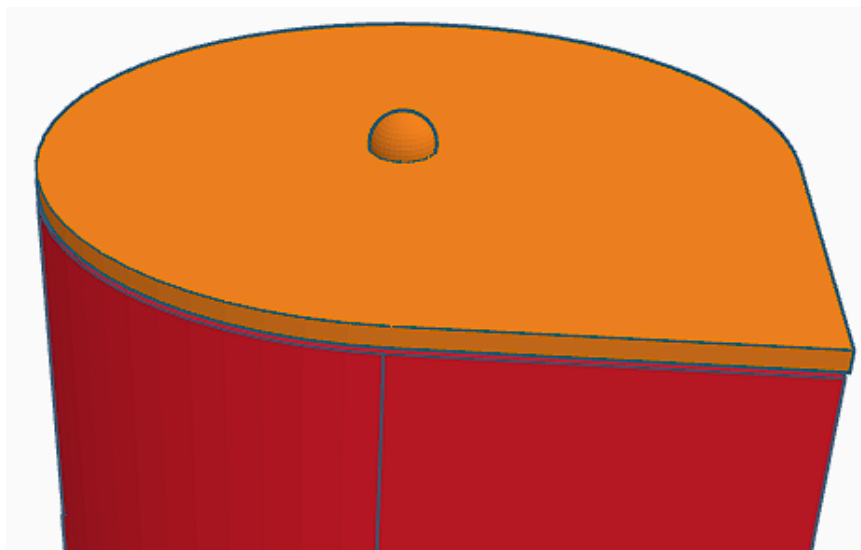


Slika 6: Pokrov za zakrivanje električnih krmilnih delov

Vir: Lasten, 2024, 5

- Drugi pokrov pa služi za zapiranje posode za hrano, kar preprečuje nenamerno dostopanje do hrane s strani hišnih ljubljencev ali drugih zunanjih dejavnikov.

(Slika 7)



Slika 7: Pokrov za zbiralno posodo

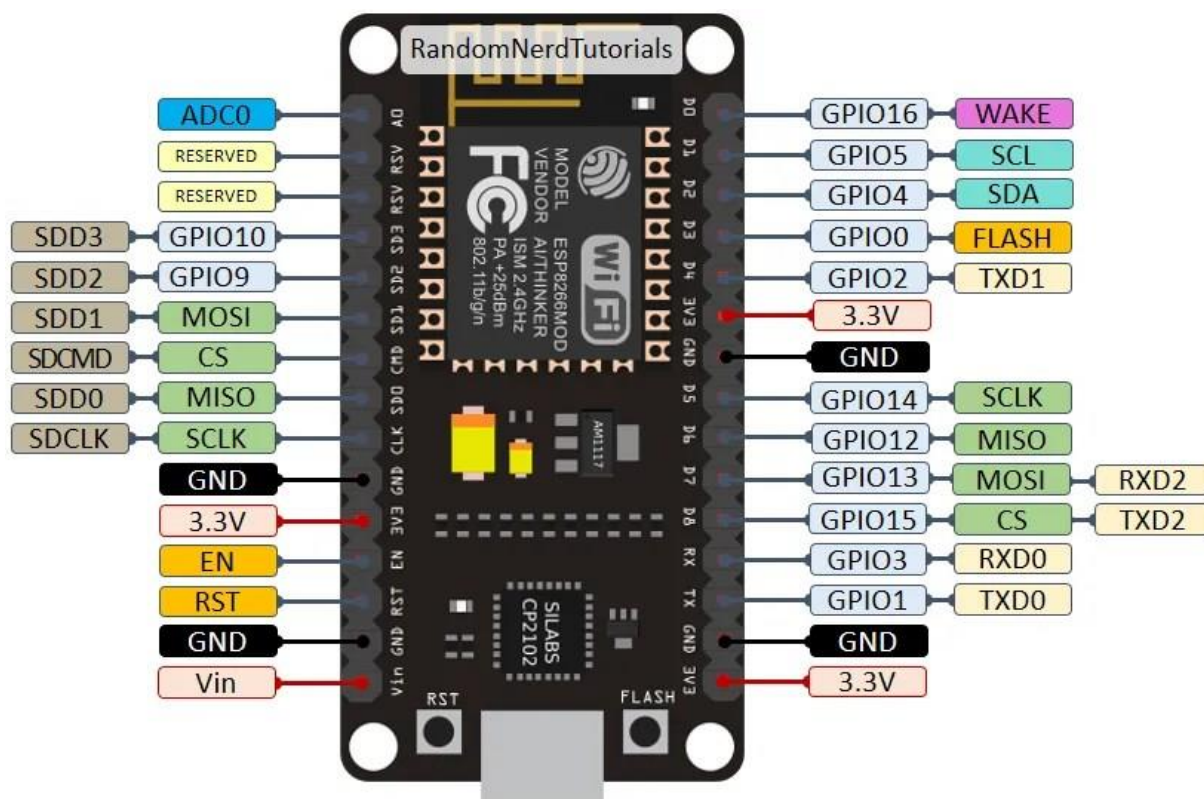
Vir: Lasten, 2024, 5

2.3 KOMPONENTE

2.3.1 NodeMCU ESP8266 (MIKROKRMILNIK)

NodeMCU ESP8266 je mikrokrmilnik, ki izstopa zaradi vgrajene brezžične povezljivosti preko Wi - Fi omrežja, idealen za projekte, kjer je potrebna komunikacija do naprave na daljavo. Ima zmogljiv 32-bitni procesor, ki omogoča hitro izvajanje programov.

(Slika 8)



Slika 8: Prikaz ESP8266 pinov

Vir: (<https://i0.wp.com/randomnerdtutorials.com/wp-content/uploads/2019/05/ESP8266-NodeMCU-kit-12-E-pinout-gpio-pin.png?resize=817%2C542&quality=100&strip=all&ssl=1>), 2024, 5

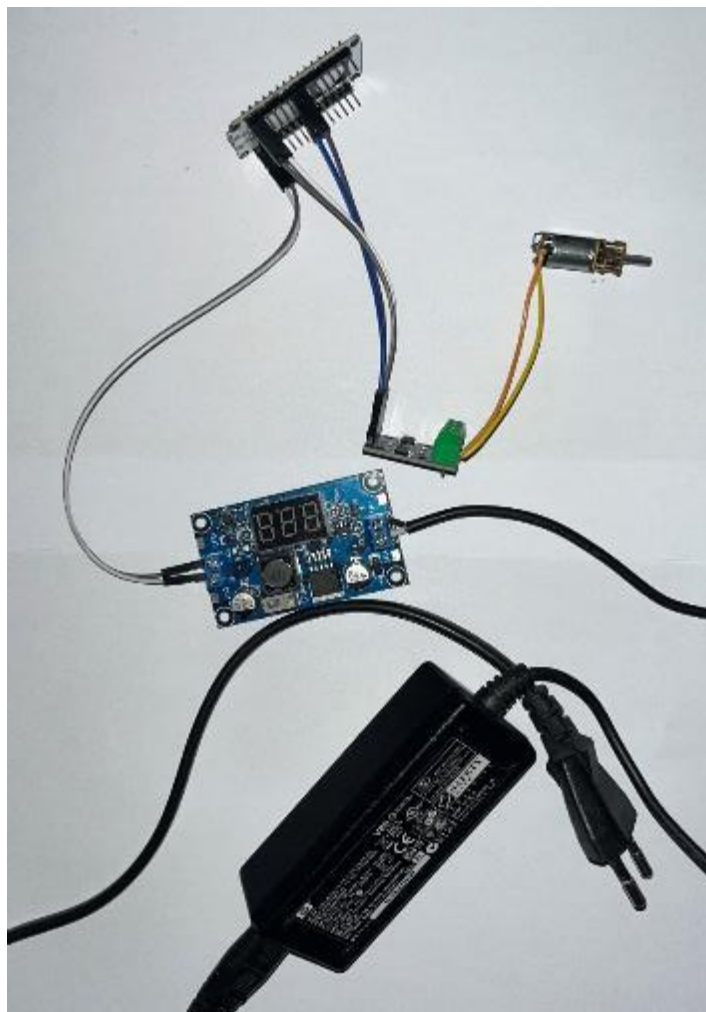
Table 1: NodeMCU ESP8266, Pini in opis pinov

Kategorija pina	Ime	Opis
Napajanje	Micro-USB, 3.3V, GND, Vin	Micro-USB: NodeMCU se lahko napaja preko vrata USB 3.3V: Reguliranih 3,3V se lahko dovaja na ta pin za napajanje vezja GND: Zemeljski pini Vin: Zunanje napajanje
Kontrolni pini	EN, RST	Pin in gumb ponastavita mikrokrmilnik
Analogni pin	A0	Uporabljen za merjenje analogne napetosti v območju 0-3,3V
GPIO pini	GPIO1 do GPIO16	NodeMCU ima 16 splošnih vhodno-izhodnih pinov na svoji plošči
SPI pini	SD1, CMD, SD0, CLK	NodeMCU ima štiri pine, ki so na voljo za SPI komunikacijo.
UART pini	TXD0, RXD0, TXD2, RXD2	NodeMCU ima dve vmesnika UART, UART0 (RXD0 in TXD0) ter UART1 (RXD1 in TXD1). UART1 se uporablja za nalaganje firmware/programa.
I2C pini		NodeMCU podpira funkcionalnost I2C, vendar zaradi interne funkcionalnosti teh pinov morate ugotoviti, kateri pin je I2C.

Vir : (<https://components101.com/development-boards/nodemcu-esp8266-pinout-features-and-datasheet>), 2024, 1

2.3.2 OSTALE KOMPONENTE

Za napajanje uporabljam polnilec za računalnik, ki zagotavlja stabilen tok pri 5V in 2000mA. Poleg tega sem dodal DC/DC pretvornik za regulacijo napetosti na izhodu za zmogljivo napetost mikrokrmilnika (3.3V), L9110S dual H-most za krmiljenje motorja ter Micro Metal Gear DC Motor s hitrostjo vrtenja 50rpm. (Slika 9)



Slika 9: Vezje komponent

Vir : Lasten, 2024, 2

2.4 ELEKTRIČNO VEZJE

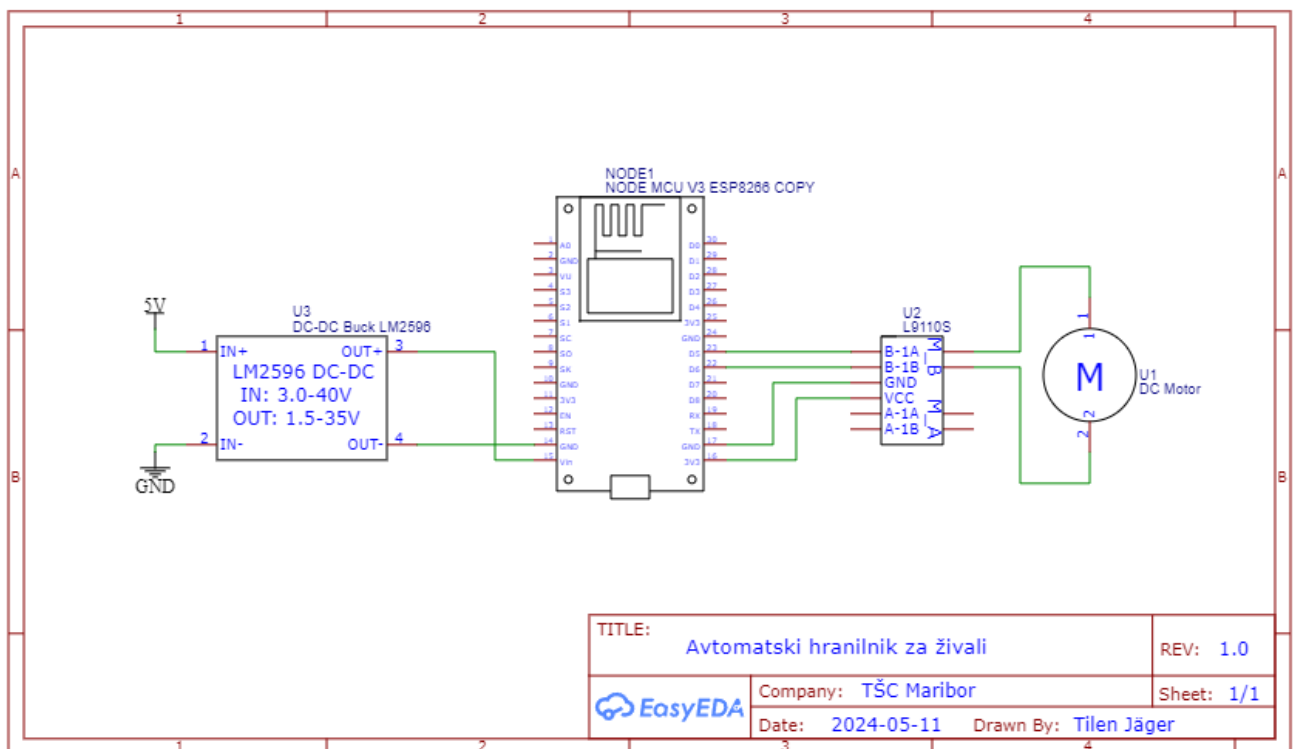
Električni načrt izvedel v programu EasyEDA. Program je preprost za uporabo, ima že v naprej definirane komponente.

Slika 10 prikazuje tok, ki se pretvarja z DC/DC pretvornikom. Napetost se zniža iz 5V na 3,3V.

Napetost potuje do mikrokrmilnika, kjer se uporablja za napajanje priključkov VIN (pozitivna napetost) in GND (negativna napetost).

Mikrokrmilnik oddaja napetost 3,3V v H - most, ki prejema napajanje.

Povezani so digitalni pini (D5 in D6) s priključki B - A1 in B - 1B. Ti prenašajo ukaze za kontroliranje DC motorja. Izhod iz M_B (H - Most) usmerjamo v DC motor.



Slika 10: Električni načrt

Vir : Lasten, 2024, 5

2.5 PROGRAMSKA KODA

Za uspešno izvedbo projekta bodo potrebni dve različni programski kodi:

Arduino Koda:

Je glavna koda, napisana v Arduino IDE programu, ki bo osredotočena za upravljanje motorja in branje svetovnega časa. Zagotovila bo pravilno delovanje mehanizma za dovajanje hrane.

Flutter Koda:

Se bo osredotočila na komuniciranje med ukazi lastnika in opravljanje mehanizma. Ukazi bodo možni preko uporabe mobilne aplikacije katera bo enostavna za uporabo.

2.6 PROGRAMIRANJE MIKROKRMILNIKA

V kodi sem vključil vse potrebne knjižnice. Za projekt sem potreboval knjižnico za vzpostavitev brezžične povezave, izdelovanje in prikaz spletne strani in knjižnico za prepoznavanje časa. (Slika 11)

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <time.h>
```

Slika 11: Vključene knjižnice

Vir: Lasten, 2024, 5

V nadaljevanju je vpisana koda za povezovanje z brezžičnim internetom. Pod spremenljivke je bilo ustrezno zapisati ime in geslo lokalnega omrežja, na katerega bo mikrokrmilnik povezan. (Slika 12)

```
const char* ssid = "Ime WiFi-ja";
const char* password = "Geslo";
```

Slika 12: Prikaz SSID in gesla

Vir: Lasten, 2024, 5

Za krmiljenje motorja sem razdelil kodo na 3 dele in sicer na:

- Start (kadar pritisneš gum Start se motor začne vrteti) (Slika 13),

```
//Ob aktivaciji "Start" gumba se motor zažene
void Start() {
    digitalWrite(D5, HIGH);
    digitalWrite(D6, LOW);
    motorRunning = true;
    handleRoot();
}
```

Slika 13: Prikaz dela kode "Start"

Vir: Lasten, 2024, 5

- Stop (kadar pritisneš gum stop se motor neha vrteti) (Slika 14) in

```
//Ob aktivaciji "Stop" gumba se motor ustavi
void Stop() {
    digitalWrite(D6, HIGH);
    digitalWrite(D5, HIGH);
    motorRunning = false;
    handleRoot();
}
```

Slika 14: Prikaz dela kode "Stop"

Vir: Lasten, 2024, 5

- Nastavitev Ure (kadar je nastavljena ura enaka svetovnemu času se motor začne vrteti). (Slika 15)

```
void loop() {
    //Ob izenacenju izbranega časa z Svetovnim časom se motor začne vrteti
    if ((p_tm->tm_hour == selectedHour) && (p_tm->tm_min == selectedMinute)) {
        digitalWrite(D5, HIGH);
        digitalWrite(D6, LOW);
        delay(4000);
        digitalWrite(D6, HIGH);
        digitalWrite(D5, HIGH);
        delay(56000);
    }
}
```

Slika 15 Prikaz dela kode "Nastavitev ure"

Vir: Lasten, 2024, 5

Za komunikacijo mikrokrmilnika z mobilno napravo sem moral naredil spletno stran v Arduino kodi s katero lahko dostopaš preko IP naslova in je možno opravljati na enak način kot mobilno aplikacijo. (Slika 16)

DIY Pet Feeder

Motor status: Stopped

Start

Set Time

Hour:
Minute:
Submit

Slika 16: Prikaz izgleda aplikacije S Arduino kode

Vir: Lasten, 2024, 2

(Celotna koda je podana v prilogi A)

2.6.1 PROGRAMSKA OPREMA ESP 8266 V ARDUINU

V tem navodilu bom korak za korakom razložil postopek namestitve knjižnice ESP8266 v razvojnem okolju Arduino IDE, ki omogoča programiranje za mikrokrmilnik ESP8266.

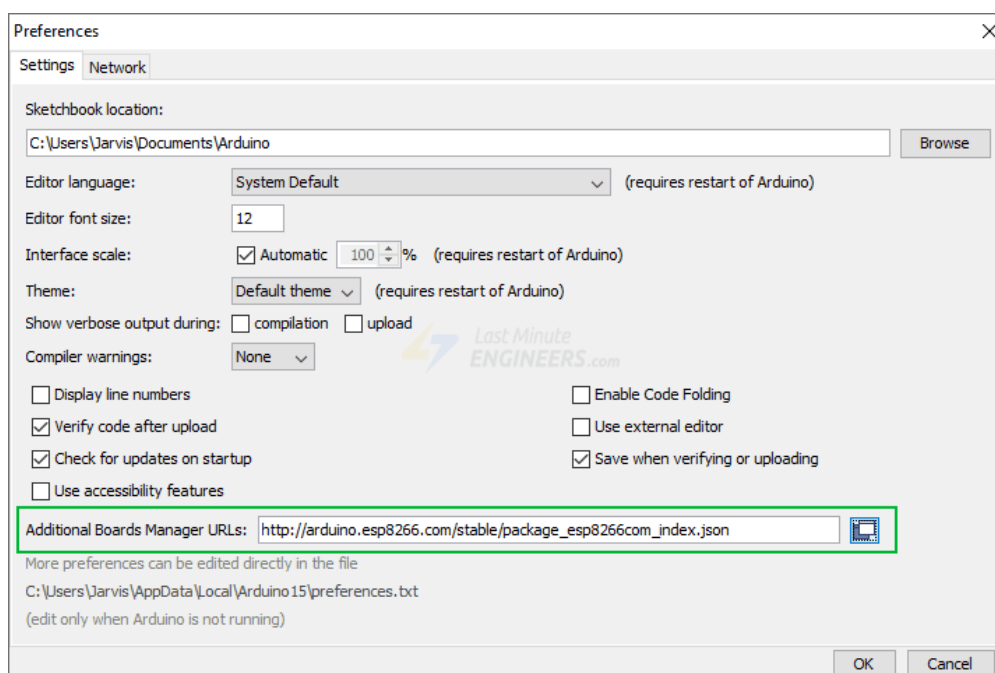
Zaženite Arduino IDE in se premaknite na Datoteka > Nastavitve (Slika 17).



Slika 17: Prikaz ročnega nalaganja knjižnice

Vir: <https://lastminuteengineers.com/wp-content/uploads/iot/Arduino-IDE-Preferences.png>, 2024, 5

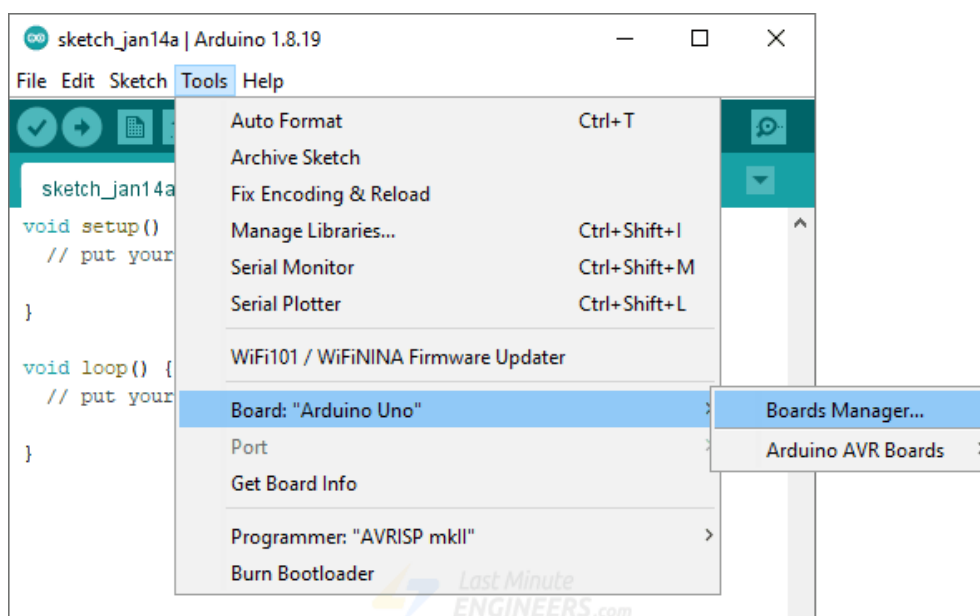
Nato pod "Additional Boards Manager URLs" vpišemo spletno stran: http://arduino.esp8266.com/stable/package_esp8266com_index.json in pritisnemo "v redu" (Slika 18).



Slika 18: Prikaz vpisa spletne strani

Vir: <https://lastminuteengineers.com/wp-content/uploads/iot/Arduino-IDE-ESP8266-Json-URL.png>, 2024, 5

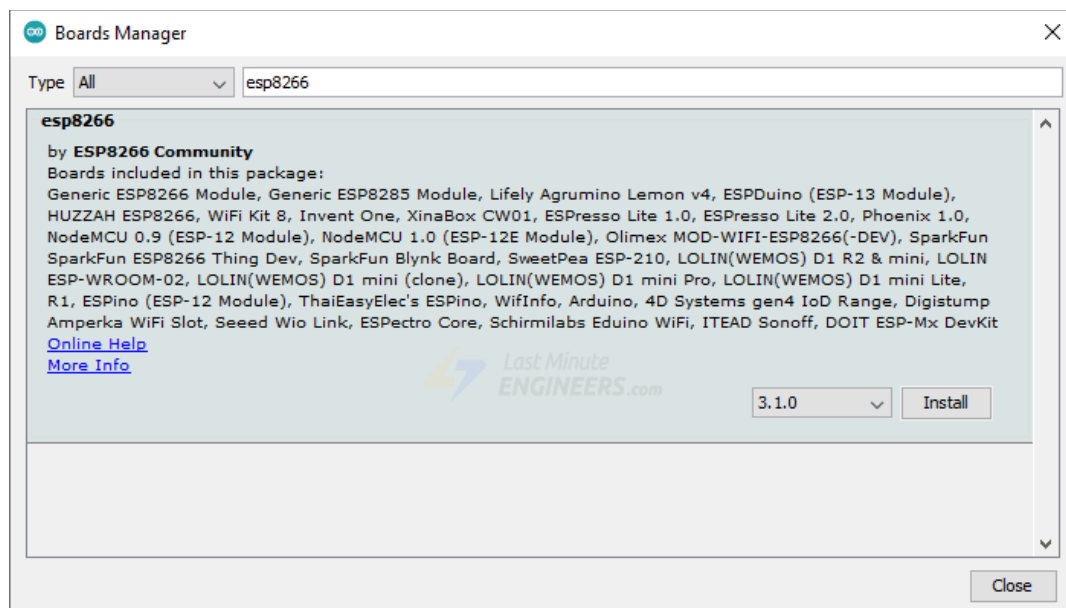
Ko smo to opravili gremo pod orodja (Tools), izberemo plošča (Board) ter pritisnemo "Boards manager" (Slika 19).



Slika 19: Prikaz izbire "boards manager"

Vir: <https://lastminuteengineers.com/wp-content/uploads/iot/Arduino-IDE-Boards-Manager.png>, 2024, 5

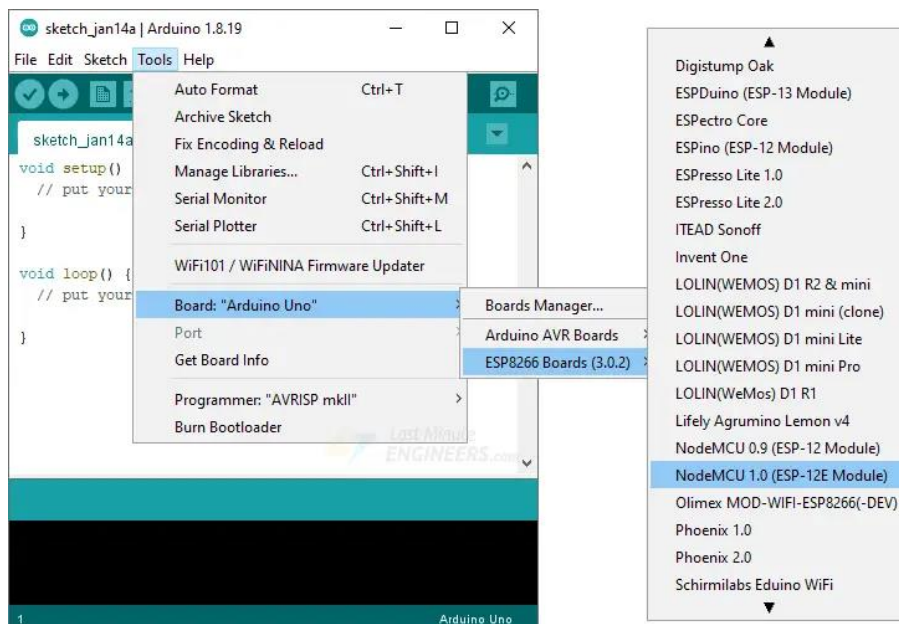
V brskalniku napišemo ESP8266 ter gumb "install" (Slika 20).



Slika 20: Prikaz instaliranja ESP8266

Vir: <https://lastminuteengineers.com/wp-content/uploads/iot/Arduino-IDE-ESP8266-Package.png>, 2024, 5

Na koncu v programu Arduino IDE izberemo "orodja", nato "plošča" in nazadnje pod zavihek "ESP8266" izberemo naš mikrokrmilnik (Slika 21).



Slika 21: Prikaz izbire plošče

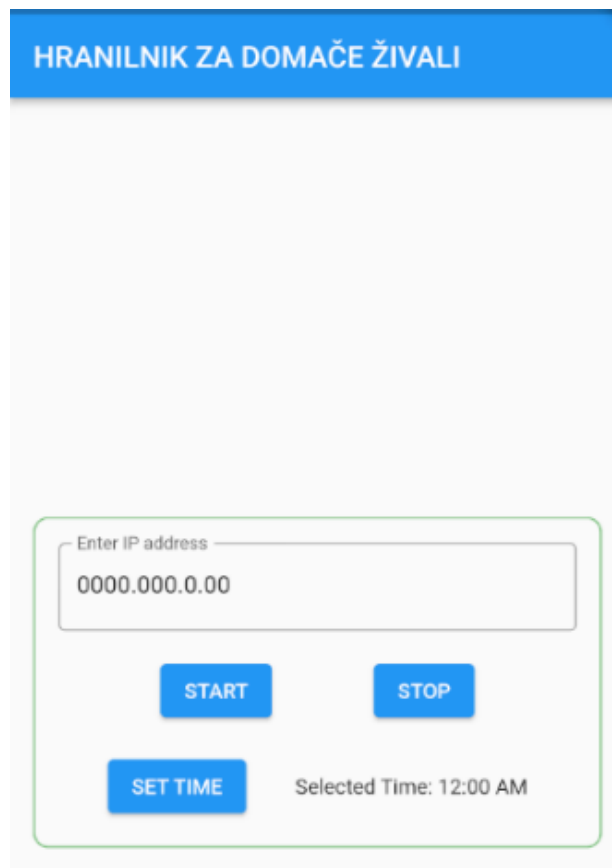
Vir: <https://lastminuteengineers.com/wp-content/uploads/iot/Arduino-IDE-Boards-Manager-ESP8266-Boards.png>, 2024, 5

2.7 PROGRAMIRANJE SPLETNE APLIKACIJE

Za izdelavo mobilne aplikacije sem se odločil uporabiti Flutter Framework, ki omogoča preprosto izdelavo mobilnih aplikacij. Aplikacija omogoča nastavitve IP naslova in samo komunikacijo z mikrokrmilnikom. Omogoča hranjenje na pritisk gumba ali nastavitve časa za naslednjo izvedbo.

Ena izmed ključnih funkcij je, da se vsi vneseni podatki shranijo ob zaprtju aplikacije. To zagotavlja, da so nastavitve (IP naslov in nastavljen čas) vedno na voljo in se ohranijo, tudi če se aplikacija zapre ali če je telefon izklopljen. (Slika 22)

(Celotna koda je podana v prilogi B)



Slika 22: Prikaz izgleda aplikacije

Vir: Lasten, 2024, 5

3 ZAKLUČEK

V maturitetni nalogi sem se lotil izdelave avtomatskega hranilnika z domače živali s pomočjo programa Arduino IDE in Flutter.

Projekt sam po sebi izboljšuje vsakodnevni problem, s katerim se srečujejo lastniki hišnih ljubljencev. Lastnikom omogoča brezskrbno hranjenje svojih ljubljencev, ko jih ni doma, bodi si zaradi službe ali daljšega potovanja.

Za izdelovanje tega projekta potrebujete osnovno znanje programskega jezika C++ in Dart, ki se uporablja za Flutter aplikacijo.

V začetku pisanja diplomske naloge sem si zastavil tri hipoteze.

Hipoteza ena se je glasila: »Uporabljanje avtomatskega hranilnika za domače živali preko aplikacije je enostavna za uporabo.« Hipotezo potrjujem, saj je aplikacija narejena na enostaven način, ki omogoča uporabo vsakomur, tako mladi, srednji kot starejši generaciji.

Hipoteza dva se glasi: »S uporabo avtomatskega hranilnika za domače živali lastniku omogoča prihranek časa in zmanjšuje stopnjo skrbi.« To hipotezo potrjujem, saj uporaba avtomatskega hranilnika dejansko lastnikom omogoča prihranek časa pri rednem hranjenju domačih živali, saj ni potrebno ročno dozirati hrane.

Hipoteza tri se glasi: »Uporaba hranilnika ni odvisna od lokacije lastnika.« Hipotezo ne potrjujem, ker trenutno aplikacija deluje samo znotraj istega omrežja.

Projekt bom v prihodnjem izboljšal:

- Dodal bom podatkovno bazo, z katero bom omogočal upravljanje z napravo, ki ni omejena na lokacijo uporabnika,
- Z namestitvijo senzorja za težo, katera bo kontrolirala količino hrane, ki jo lahko nastavljaš v mobilni aplikaciji in
- Video nadzor za preverjanje količine hrane v posodi.

4 VIRI

[1] Arduino. Spletna stran platforme Arduino. (online). 2024. (Citirano 11.5.2024).

Dostopno na : <https://www.arduino.cc/en/software>

[2] Components101. NodeMCU ESP8266. (online). 2024. (Citirano 11.05.2024).

Dostopno na: <https://components101.com/development-boards/nodemcu-esp8266-pinout-features-and-datasheet>

[3] EasyEDA. Spletna stran podjetja EasyEDA, program EasyEDA. (online). 2024. (Citirano 11.05.2024).

Dostopno na: <https://easyeda.com/editor>

[4] Flutter. Spletna stran podjetja Flutter, program Flutter. (online). 2024 (Citirano 11.05.2024).

Dostopno na: <https://flutter.dev/>

[5] Last Minute Engineer. Insight into ESP8266 NodeMCU features & using it with Arduino IDE. (online). 2024 (Citirano 11.05.2024).

Dostopno na: <https://lastminuteengineers.com/esp8266-nodemcu-arduino-tutorial/>

[6] Tinkercad. Spletna stran podjetja Tinkercad, program Tinkercad. (online). 2024. (Citirano 11.05.2024).

Dostopno na: <https://www.tinkercad.com/dashboard>

5 PRILOGE

PRILOGA A: Arduino koda

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <time.h>

// Definiranje SSID-ja in gesla za WiFi povezavo
const char* ssid = "Ime WiFi-ja";
const char* password = "Geslo";

// Ustvarjanje objekta strežnika na portu 80
ESP8266WebServer server(80);

// Nastavitve časovnega pasu
int timezoneOffset = 2 * 3600; // Časovni zamik v sekundah
int dst = 0; // Poletni čas (DST) - 0 pomeni neaktiven

// Globalne spremenljivke za shranjevanje izbranega časa, stanja motorja in
// HTML oblikovanja
int selectedHour = 0;
int selectedMinute = 0;
bool motorRunning = false;
String header = "<head><center><h1>DIY Pet Feeder</h1></center></head>";
String status_page_stop = "<body><center><p>Motor status: Stopped</p>";
String status_page_start = "<body><center><p>Motor status: Started</p>";
String button_start = "<p><a
href=\"Start\"><button>Start</button></a></p></center></body>";
String button_stop = "<p><a
href=\"Stop\"><button>Stop</button></a></p></center></body>";

// Obdelovalna funkcija za zahtevo na korenskem URL-ju
void handleRoot() {
    // Sestavljanje HTML strani
    String html = "<html><body>";
    html += header;

    // Dodajanje statusa motorja in gumbov za upravljanje motorja
    html += motorRunning ? status_page_start : status_page_stop;
    html += motorRunning ? button_stop : button_start;

    // Dodajanje obrazca za nastavitve časa
    html += "<h1>Set Time</h1>";
    html += "<form method='post' action='/settime'>";
    html += "Hour: <input type='number' name='hour' min='0' max='23'><br>";
}
```

```

html += "Minute: <input type='number' name='minute' min='0' max='59'><br>";
html += "<input type='submit' value='Submit'>";
html += "</form>";

html += "</body></html>";

// Pošiljanje odgovora s sestavljeno HTML stranjo
server.send(200, "text/html", html);
}

// Obdelovalna funkcija za nastavitev časa preko HTTP POST zahteve
void handleSetTime() {
    // Preverjanje, ali so bili prejeti podatki o uri
    if (server.args() > 0) {
        // Prehod skozi prejete argumente
        for (uint8_t i = 0; i < server.args(); i++) {
            // Preverjanje imen argumentov in posodabljanje izbranega časa
            if (server.argName(i) == "hour") {
                selectedHour = server.arg(i).toInt();
            } else if (server.argName(i) == "minute") {
                selectedMinute = server.arg(i).toInt();
            }
        }
    }
    // Pošiljanje odgovora s sporočilom o uspešni nastavitvi časa
    server.send(200, "text/plain", "Time set successfully!");
}

// Funkcija za zagon motorja
void Start() {
    digitalWrite(D5, HIGH);
    digitalWrite(D6, LOW);
    motorRunning = true;
    handleRoot(); // Ponovno pošiljanje korenske strani
}

// Funkcija za ustavitev motorja
void Stop() {
    digitalWrite(D6, HIGH);
    digitalWrite(D5, HIGH);
    motorRunning = false;
    handleRoot(); // Ponovno pošiljanje korenske strani
}

// Inicializacija kode

```

```

void setup() {
  // Nastavitev pinov kot izhodov
  pinMode(D5, OUTPUT);
  pinMode(D6, OUTPUT);

  Serial.begin(115200); // Začetek serijske komunikacije

  // Povezovanje z WiFi omrežjem
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.println("Connecting to WiFi...");
  }
  Serial.println("WiFi connected");

  // Nastavitev obdelovalnih funkcij za določene URI-je
  server.on("/", handleRoot);
  server.on("/settime", handleSetTime);
  server.on("/Start", Start);
  server.on("/Stop", Stop);

  server.begin(); // Začetek strežnika
  Serial.println("HTTP server started");
  Serial.print("NodeMCU IP Address: ");
  Serial.println(WiFi.localIP());

  // Sincronizacija časa z NTP strežniki
  configTime(timezoneOffset, dst, "pool.ntp.org", "time.nist.gov");
  Serial.println("\nWaiting for Internet time");
  // Čakanje na sinhronizacijo časa
  while (!time(nullptr)) {
    Serial.print("*");
    delay(1000);
  }

  Serial.println("\nTime response....OK");
}

// Glavna zanka programa
void loop() {
  server.handleClient(); // Obdelovanje odjemalčevih zahtev

  // Preverjanje časa in delovanje motorja ob doseženem času
  time_t now = time(nullptr);
  struct tm* p_tm = localtime(&now);
  if ((p_tm->tm_hour == selectedHour) && (p_tm->tm_min == selectedMinute)) {
    digitalWrite(D5, HIGH);
  }
}

```

```
    digitalWrite(D6, LOW);  
    delay(4000);  
    digitalWrite(D6, HIGH);  
    digitalWrite(D5, HIGH);  
    delay(56000);  
  }  
}
```

Vir: Lasten, 2024, 2

PRILOGA B: Flutter koda

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Pet Feeder Control',
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: false,
      ),
      home: const MyHomePage(),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({Key? key}) : super(key: key);

  @override
  _MyHomePageState createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  final Future<SharedPreferences> _prefs = SharedPreferences.getInstance();
  final TextEditingController _ipController = TextEditingController();
  late String _ipAddress;
  late String _baseUrl;
  late TimeOfDay? _selectedTime = null;

  @override
  void initState() {
    _prefs.then((SharedPreferences prefs) {
      setState(() {
        _ipAddress = prefs.getString('ip') ?? '000.000.000.000'; //IP adras
        _baseUrl = 'http://$_ipAddress';
        _ipController.text = _ipAddress;
      });
    });
  }
}
```

```

        _selectedTime = new TimeOfDay(hour: prefs.getInt('time_hour') ?? 0,
minute: prefs.getInt('time_minute') ?? 0);
    });
    });
    super.initState();
}

Future<void> _sendCommand(String command) async {
  try {
    final response = await http.get(Uri.parse('$_baseUrl/$command'));
    if (response.statusCode == 200) {
      print('Command sent: $command');
    } else {
      print('Failed to send command: $command');
    }
  } catch (e) {
    print('Error: $e');
  }
}

Future<void> _setFeedingTime(TimeOfDay time) async {
  final response = await http.post(
    Uri.parse('$_baseUrl/settime?hour=${time.hour}&minute=${time.minute}'),
  );
  if (response.statusCode == 200) {
    print('Time set successfully');
  } else {
    print('Failed to set time');
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('HRANILNIK ZA DOMAČE ŽIVALI'),
    ),
    body: Center(
      child: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Container(
              padding: const EdgeInsets.all(16.0),
              decoration: BoxDecoration(
                border: Border.all(color: Colors.green),

```

```

        borderRadius: BorderRadius.circular(10),
      ),
      child: Column(
        children: [
          TextFormField(
            controller: _ipController,
            keyboardType: TextInputType.number,
            decoration: InputDecoration(
              labelText: 'Enter IP address',
              border: OutlineInputBorder(),
              errorText: _ipController.text.isNotEmpty &&
!_isValidIP(_ipController.text) ? 'Enter a valid IP address' : null,
            ),
            onChanged: (value) async {
              final SharedPreferences prefs = await _prefs;
              setState(() {
                prefs.setString('ip', value.toString());
                _ipAddress = value;
                _baseUrl = 'http://$_ipAddress';
              });
            },
          ),
          const SizedBox(height: 16),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              ElevatedButton(
                onPressed: () => _sendCommand('Start'),
                child: const Text('START'),
              ),
              ElevatedButton(
                onPressed: () => _sendCommand('Stop'),
                child: const Text('STOP'),
              ),
            ],
          ),
          const SizedBox(height: 16),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              ElevatedButton(
                onPressed: () async {
                  final TimeOfDay? pickedTime = await
showTimePicker(
                    context: context,
                    initialTime: _selectedTime!,
                  );

```

```

        if (pickedTime != null) {
            final SharedPreferences prefs = await _prefs;
            prefs.setInt('time_hour', pickedTime.hour);
            prefs.setInt('time_minute', pickedTime.minute);
            setState(() {
                _selectedTime = pickedTime;
            });
            _setFeedingTime(pickedTime);
        }
    },
    child: const Text('SET TIME'),
),
Text(
    'Selected Time: ${_selectedTime?.format(context)}',
),
],
),
],
),
),
],
),
),
),
);
}

bool _isValidIP(String value) {
    final List<String> splitValues = value.split('.');
    if (splitValues.length != 4) return false;
    return splitValues.every((element) {
        final intValue = int.tryParse(element);
        return intValue != null && intValue >= 0 && intValue <= 255;
    });
}
}

```

Vir: Lasten, 2024, 5